

Technical Deep Dive (Sanitized Public Version)

Version: 2025-12-31

Contents

1. Executive summary
2. System architecture and dataflow
3. Event pipeline: stages and design rationale
4. HopGraph: data model and attack reconstruction
5. Explainable triage: T1/T2 LLM integration
6. SBOM runtime fusion
7. Operational considerations (scaling, storage, observability)
8. Security, privacy, and compliance
9. Demo & reproducibility
10. Roadmap and next steps

1. Executive summary

JanuSec is an AI-assisted pre-SIEM triage platform designed to reduce alert noise and accelerate investigation.

2. System architecture and dataflow

At a high level, JanuSec ingests telemetry from multiple domains (endpoint, network, cloud, identity).

Core components (public view):

- Ingest connectors: cloud CSPM, EDR adapters, network collectors, email parsers. Connectors normalize data into a common schema.
- Enrichment: geoip, ASN, threat intel lookups, vulnerability mappings and SBOM associations.
- Event pipeline: 30 stages (lightweight → heavy) that add factors and scores to events.
- HopGraph: a graph layer that models entities (host, user, file, ip, domain, process, vulnerability).
- Triage engine: T1 (fast triage) and T2 (deep investigation) outputs, persona-aware.
- Storage: tiered hot/warm/cold for event and evidence retention; artifacts stored per incident in a structured format.

Diagram (logical): Ingest → Enrich → Pipeline → HopGraph → Triage → Storage

3. Event pipeline: stages and design rationale

The pipeline is implemented as 30 modular stages. Stages are intentionally small, single-responsible.

Sanitized stage categories:

- Baseline & parsing: normalization, schema validation, timestamp alignment.
- Enrichment: geoip, user lookup, host asset mapping.
- Signature & heuristic checks: regex-based indicators, LOLBIN patterns, common phishing indicators.
- ML/Statistical analyses: novelty detection, beacon detection, domain novelty (HEAVY).
- Malware / payload analysis: binary payload checks, SBOM execution correlation (HEAVY).
- Correlation & graph joins: mapping events into HopGraph nodes and edges.
- Dedup & quality: cluster dedupe, quality filters, coverage tracking.

Design rationale points:

- Explainability: every factor includes provenance (stage, inputs) so downstream triage and auditors can trace back.
- Modularity: stages are independent so they can be tested and maintained in isolation.
- Adaptive gating: heavy computation is skipped when earlier stages produce high-confidence results.

4. HopGraph: data model and attack reconstruction

HopGraph is the engine that joins events into an investigative graph. The public model is intentionally simplified.

Data model (public):

- Nodes: host, user, process, file, ip, domain, vulnerability
- Edges: executed_on, spawned_by, connected_to, downloaded_from, exploited_via

Graph construction steps (sanitized):

1. Event canonicalization: map raw event fields into canonical node identifiers.
2. Node creation: create nodes for distinct canonical entities.
3. Edge creation: create edges based on observed relationships in events (e.g., process executed_on
4. Path scoring: compute path likelihoods using a mixture of deterministic factors (time adjacency,
5. Explainability: annotate edges and nodes with factor provenance and timestamps to allow investiga

HopGraph produces two main outputs useful for investigations:

- An annotated path (attack reconstruction) listing nodes, edges, timestamps, and contributing facto
- A compact visualization that supports analyst interactions: expand/collapse nodes, filter by facto

5. Explainable triage: T1/T2 LLM integration

JanuSec provides two triage levels:

- T1 — Fast triage: short (30–45 lines) summary for SOC analysts with WHAT, EXPLOITABILITY, WHAT
- T2 — Deep investigation: longer (60–100 lines) persona-based report with historical context, attac

Integration pattern (public):

1. Event scored and enriched.
2. System decides whether to call LLMs based on triage score threshold and budget gating.
3. On-call, system sends structured context (factors, top graph nodes, timeline segments) to the LLM
4. LLM returns a structured textual report, which is validated against a persona schema and annotate

Gating and cost control: environment variables such as `LLM\_T1\_MIN\_TRIAGE` and `LLM_BUDGET_P

Fallback: deterministic, rule-based textual summaries are available when LLMs are unavailable or cos

6. SBOM runtime fusion

One of JanuSec's differentiators is fusing SBOM/component inventory with runtime telemetry. The publ

- Map installed components (from SBOMs) to running processes and loaded libraries.
- When an exploit pattern references a known vulnerable component, correlate the SBOM entry with run
- Annotate HopGraph with `vulnerability` nodes and link them to exploit paths.

This enables prioritization not just by CVSS but by active exploitation evidence: patching effort is

7. Operational considerations (scaling, storage, observability)

Scaling:

- Horizontal scaling for ingestion and pipeline workers via Kubernetes worker pools.
- Partitioning of graph sessions to support multi-tenant isolation.

Storage:

- Tiered storage model: hot (7 days) for immediate triage, warm (30 days) for ongoing investigations
- Evidence blobs and snapshots stored per incident under `artifacts/snapshots/{incident\_id}`.

Observability:

- Optional Prometheus metrics; code uses graceful stubs when `prometheus\_client` is not installed.
- Grafana dashboards recommended for stage latencies, skip rates, and throughput.

8. Security, privacy, and compliance

Privacy-preserving publishing:

- The public repo contains only sanitized examples and synthetic data generators.

- Do not publish real logs, hostnames, IPs, or customer data.

Compliance:

- Explainability and factor provenance help meet regulatory requirements in the EU (AI Act, GDPR Art

9. Demo & reproducibility

The repository includes a synthetic event generator and a demo matcher that illustrates a sanitized

```
```bash
python tools/data-ingestion-simulator/generate_events.py --count 100 > events.jsonl
python tools/data-ingestion-simulator/demo_matcher.py < events.jsonl
```
```

CI includes a workflow that runs the generator and matcher to ensure the demo continues to work.

10. Roadmap and next steps

Short-term priorities (public):

- Finish GCP asset inventory connector and AWS Security Hub integration.
- Implement KAPE execution detection and KAPE CSV ingestion (Options A+B from roadmap).
- Multi-provider LLM failover to improve summary reliability.

Medium-term:

- Tiered storage implementation, performance validation at 5K-10K events/sec, Okta/Azure AD deep int

Closing note

This public deep dive is intended to provide reviewers and potential partners with a clear architect

Appendix A — Example sanitized factor schema

The following is an illustrative, sanitized example of a factor object attached to an event (fields

```
...
{
  ■ "factor_id": "rare_token_001",
  ■ "stage": "rare_token",
  ■ "value": 0.87,
  ■ "description": "Login token rarely seen in historical data",
  ■ "provenance": {
    ■■ "source_event_id": "evt-12345",
    ■■ "timestamp": "2025-12-26T14:32:17Z",
    ■■ "inputs": ["user", "token_type", "historical_frequency"]
  }
}
```

Appendix B — Sample sanitized HopGraph JSON snippet

This synthetic snippet shows how nodes and edges might be represented for visualization or API consu

```
...
{
  ■ "nodes": [
    ■■ {"id": "host-001", "type": "host", "meta": {"os": "linux"}},
    ■■ {"id": "user-A", "type": "user", "meta": {}},
  ]
}
```

```
■{ "id": "process-4523", "type": "process", "meta": { "cmd": "/usr/bin/bash -c ..." } }
■],
■"edges": [
■{ "src": "user-A", "type": "executed_on", "dst": "host-001", "ts": "2025-12-26T14:31:50Z" },
■{ "src": "process-4523", "type": "spawned_by", "dst": "user-A", "ts": "2025-12-26T14:32:10Z" }
■]
}...
```

Appendix C — Implementation checklist for production readiness

- Confirm connector resilience (retries, DLQ) for each cloud/EDR connector
- Enable Prometheus metrics in production; deploy Grafana dashboards
- Harden storage and evidence access; implement tenant-based quotas and legal-hold APIs
- Add EDR snapshot orchestration under strict RBAC and consent rules

References

- Public research on graph-based attack reconstruction: [link placeholder]
- MITRE ATT&CK: <https://attack.mitre.org/>
- EU AI Act / GDPR requirements (public guidance): [link placeholder]

Acknowledgements

This public document was prepared by the JanuSec engineering and product team. For private follow-up